Faculty of Engineering
School of Electrical Engineering

Course Number : 0512.4490
Course Name : Advanced Computer Architecture Lab
Course Year: 2026

Lab #2: ASIC Hierarchical Verification: Low Level Simulator

Student Name: Itay Zohar
Student ID: 209276633

Student Name: Emad Mazzawi
Student ID: 314744996

Question 1 - Understanding Registers Simulation

1. After execution of 2 cycles the registers will hold the following values -
   $r[2] = 12$
   $r[3] = 45$
   $r[4] = 57$
   $r[5] = -12$

2. In the given command, there are 2 issues:
   - We write to spro->r[1] which represents the "old" value of the register. These values should only be read.
   - We read from sprn->r[3] which represents the "new" value of the register. These values should only be written to.

Question 2 - Understanding Micro-architecture

1. Each instruction goes through 7 states in this order - IDLE -> FETCH0 -> FETCH1 -> DEC0 -> DEC1 -> EXEC0 -> EXEC1. However IDLE only occurs at the start or at HALT, meaning for n instructions we will need 6n+1 clock cycles.
2. A proposed microarchitecture can support memory access every clock cycle by either using an asynchronous SRAM, or by dividing the READ operation, so that the request is issued and sampled at the same cycle, rather than in the subsequent cycle.
3. The advantages and disadvantages of this micro-architecutre are as follows -
   - It has a **simpler design**, with less control logic and easier timing analysis.
   - It is **less hardware complex**, with fewer hazards, no forwarding logic, and no pipeline stall handling.

   On the other hand it has the following disadvantages -
   - Much **lower performance**, since only one instruction progresses at once, and no overlap between the different potential pipeline stages (decode, fetch, execute, etc), this means the CPI is much lower.
   - A lot of **hardware resources are underutilized**, due to the same issue presented above.
   - The clock speed is likely lower, since the critical path is longer compared to a pipeline processor.

Question 6 - Background DMA (direct memory access) Copy

The DMA is designed as follows -
First, we introduce 2 new opcodes:

- DMA_START = 10: reads src address from r[src0], dst address from r[dst], length from r[src1]
- DMA_POLL = 11: writes 0 (done) or 1 (busy) into r[dst]

We then add these dedicated registers to **sp_registers_s**:

- dma_state (2 bit)
- dma_src (16 bit)
- dma_dst (16 bit)
- dma_len (16 bit)
- dma_data (32 bit)

Finally, we will use the following state machine for the DMA operation -

- **DMA_IDLE (0)**: doing nothing, poll returns 0
- **DMA_ISSUE (1)**: issue read from dma_src – stall if CPU is using memory
- **DMA_SAMPLE (2)**: sample result into dma_data
- **DMA_WRITE (3)**: write dma_data to dma_dst – stall if CPU is using memory, then decrement dma_len, increment dma_src and dma_dst, loop back to DMA_ISSUE or go to DMA_IDLE if dma_len hits 0

This design was added to sp.c as instructed in the question.

Question 7 - DMA testing

The assembly code is as follows：

```
   /* Setup and DMA Execution */
   asm_cmd(ADD, 5, 1, 0, 50);          // Set source base: R5 = 0x32
   asm_cmd(ADD, 6, 1, 0, 60);          // Set dest base: R6 = 0x3C
   asm_cmd(DMA_START, 0, 5, 6, 50);    // Trigger DMA: copy 50 words from
R5 to R6
   asm_cmd(DMA_POLL, 4, 0, 0, 0);      // Read DMA status register into R4
   asm_cmd(JEQ, 0, 4, 0, 11);          // Exit loop if R4 (idle flag) == 0

   /* Background CPU Work (Memory Stress) */
   asm_cmd(ADD, 5, 1, 0, 400);         // Pointer to stress addr 400
   asm_cmd(LD, 6, 0, 5, 0);            // Fetch value from MEM[400]
   asm_cmd(ADD, 6, 6, 1, 1);           // Increment value locally
   asm_cmd(ADD, 3, 1, 0, 401);         // Pointer to stress addr 401
   asm_cmd(ST, 0, 6, 3, 0);            // Commit incremented value to
MEM[401]
   asm_cmd(JEQ, 0, 0, 0, 3);           // Loop back to DMA_POLL

   /* Verification Logic */
   asm_cmd(ADD, 2, 1, 0, 1);           // Default result: R2 = 1 (PASS)
   asm_cmd(ADD, 3, 1, 0, 60);          // Pointer to DMA output (0x3C)
   asm_cmd(ADD, 4, 1, 0, 300);         // Pointer to golden reference
(0x12C)
   asm_cmd(ADD, 5, 1, 0, 50);          // Initialize loop counter (N=50)

   /* Loop: Comparison */
   asm_cmd(LD, 6, 0, 3, 0);            // Load actual value from output
buffer
   asm_cmd(LD, 7, 0, 4, 0);            // Load expected value from ref
buffer
   asm_cmd(JNE, 0, 6, 7, 23);          // Branch to fail if actual !=
expected
   asm_cmd(ADD, 3, 3, 1, 1);           // Post-increment output pointer
   asm_cmd(ADD, 4, 4, 1, 1);           // Post-increment reference pointer
   asm_cmd(ADD, 5, 5, 1, -1);          // Decrement loop counter
   asm_cmd(JNE, 0, 5, 0, 15);          // Loop if counter > 0

   /* Termination States */
```

```
asm_cmd(HLT, 0, 0, 0, 0);            // Normal termination (PASS)
asm_cmd(ADD, 2, 0, 0, 0);            // Result update: R2 = 0 (FAIL)
asm_cmd(HLT, 0, 0, 0, 0);            // Error termination (FAIL)

/* Initialize memory buffers */
for (i = 0; i < 50; i++) {
    mem[50 + i] = 0x1000 + i;   // Populate source data
    mem[300 + i] = 0x1000 + i;  // Populate reference data
}

mem[400] = 0x00000005;               // Init stress source
mem[401] = 0x00000000;               // Init stress sink
last_addr = 402;                     // Set dump range
```